



Security Audit

Diffuse Protocol “Adapter Updates” (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Status Definitions	13
Audit Findings	14
Centralisation	16
Conclusion	17
Our Methodology	18
Disclaimers	20
About Hashlock	21

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Diffuse Protocol team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Diffuse Prime is a non-custodial protocol that enables hedge funds to amplify yields by up to 3x through overcollateralized loans funded by Diffuse Prime LPs.

LP capital is deployed into hedge fund DeFi strategies with lending-market-grade security, enhanced by verifiable risk assessment and an automated capital protection layer.

Diffuse Protocol powers Diffuse Prime with a trustless, multichain infrastructure built on TEE, zkTLS Oracles, Collateral Abstraction, and zkMessaging, providing cryptographically verifiable trust and security.

Project Name: Diffuse Protocol

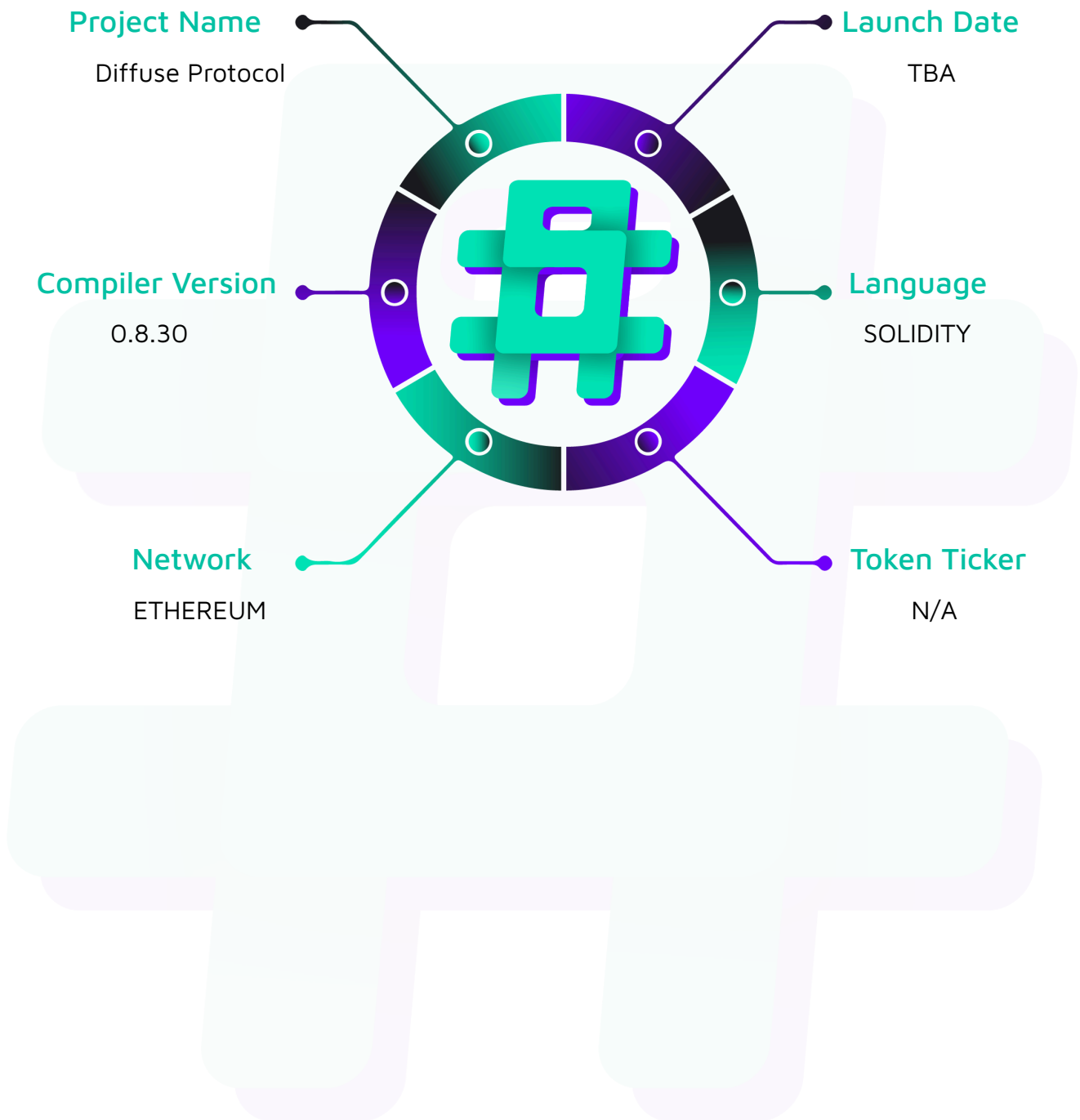
Project Type: DeFi

Compiler Version: 0.8.30

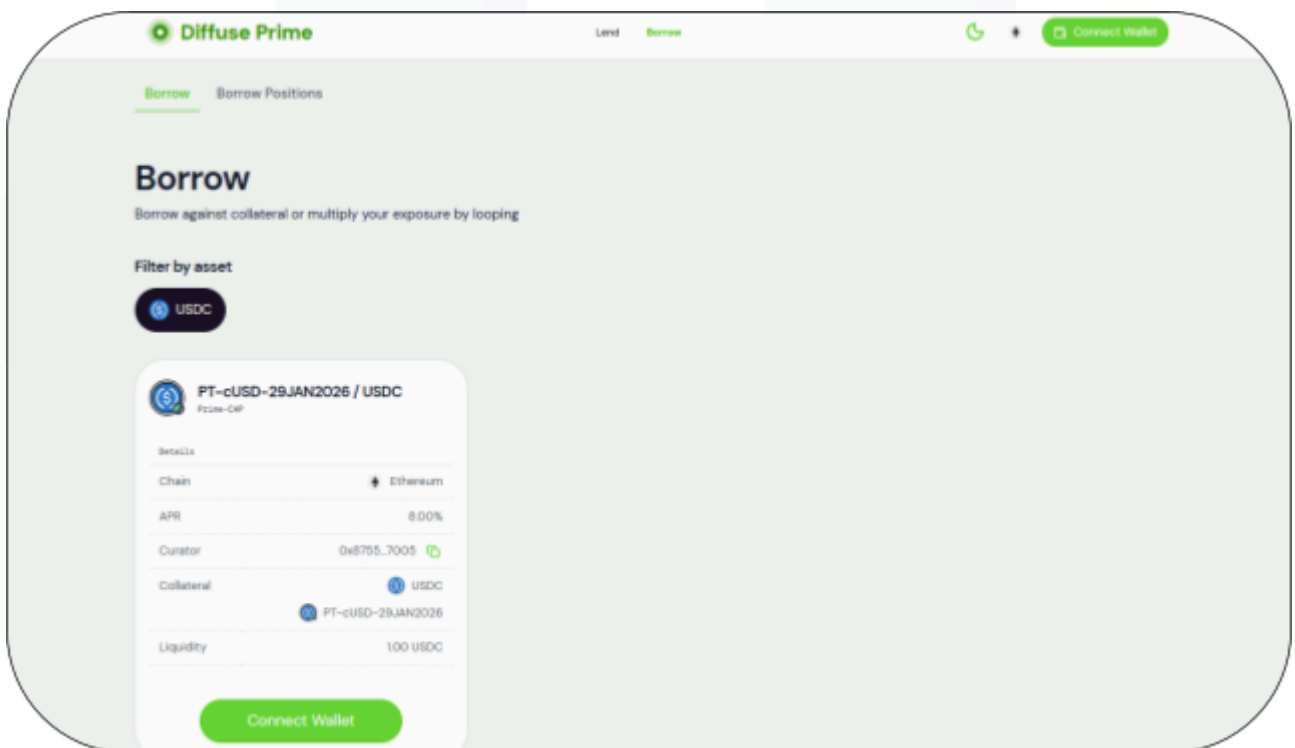
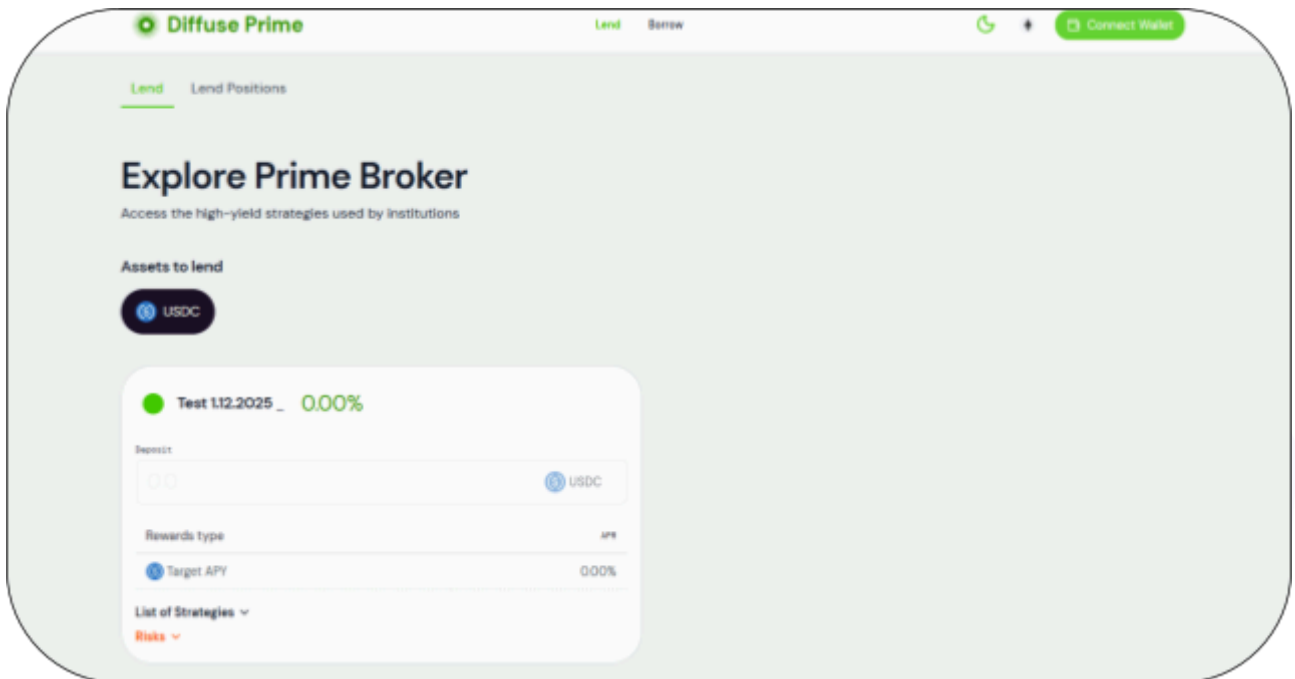
Website: app.prime.diffuse.fi

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Diffuse Protocol project. The scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Diffuse Smart Contract
Platform	Ethereum / Solidity
Audit Date	January, 2026
Contract 1	AdapterDataExtraction.sol
Contract 2	AegisMintAdapter.sol
Aegis mint adapter implementation	0xc45AE77c1A44B29831E666e35563f063bE7A76FC
Contract 3	AegisRedeemAdapter.sol
Aegis redeem adapter implementation	0x06eED4874BfACc92Baeb832996522CD1Ca1A6a87
Contract 4	PendleAdapter.sol
Pende Implementation	0x6cAC93a1469528Cb25F2d58472d450532C71d961
Pendle Proxy 1 (cUSD)	0x2121efB4c7cf0B1ccc52463B91A538b4E803b9aE
Pendle Proxy 2 (stcUSD)	0x9a9eA2FA0585Ebbe5264cD633E586e29b0a50663
Audited GitHub Commit Hash	8cd81e52f723f6ef66e36079ef89b02c962ed416
Fix Review GitHub Commit Hash	8cd81e52f723f6ef66e36079ef89b02c962ed416

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been acknowledged.

Hashlock found:

2 QAs

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
src/adapters/AdapterDataExtraction.sol - Allows adapters to extract specific configuration data - Validates encoded data length and structure - Enforces type safety using unique adapter type hashes - Handles optional extraction (returning empty bytes)	Contract achieves this functionality.
src/adapters/Aegis/AegisMintAdapter.sol - Operator: Whitelist vaults, emergency withdraw, schedule upgrades - Users: Mint Aegis tokens (yUSD), execute atomic minting with slippage protection	Contract achieves this functionality.
src/adapters/Aegis/AegisRedeemAdapter.sol - Operator: Manage instances, whitelist vaults, and emergency withdraw - Users: Initiate redemptions, finalize async swaps, retry/refund failed requests	Contract achieves this functionality.
src/adapters/PendleAdapter.sol - Whitelist vaults authorized to call the adapter - Perform emergency withdrawals of uncommon tokens - Schedule and execute time-locked upgrades or arbitrary calls	Contract achieves this functionality.

- | | |
|---|--|
| <ul style="list-style-type: none">- Swap underlying tokens for Pendle Principal Tokens (PT)- Swap/Redeem Pendle Principal Tokens (PT) for underlying tokens- Automatically handle expired PT redemption- Execute swaps with limit orders and slippage protection | |
|---|--|

Code Quality

This audit scope involves the smart contracts of the Diffuse Protocol project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality. While the contracts are upgradeable, any upgrade is subject to a **mandatory and fixed 3-day timelock**

Audit Resources

We were given the Diffuse Protocol project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore, it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

QA

[Q-01] PendleAdapter#_buy - Prefer Dedicated Buy/Sell Adapters Over IS_BUY Branching

Description

PendleAdapter currently uses the `IS_BUY` flag to switch between buy and sell flows inside a single `_buy` implementation. This increases cognitive load for reviewers and operators because `TOKEN_IN/TOKEN_OUT` semantics invert based on a mutable deployment choice. It also raises the risk of misconfiguration during initialization, where a wrong token ordering or flag leads to incorrect swaps. In addition, the runtime branching adds extra conditional checks to every swap, which is a small but recurring gas overhead compared to dedicated adapters.

Recommendation

Split the adapter into two dedicated contracts (e.g., `PendleBuyAdapter` and `PendleSellAdapter`) with fixed token directions and no runtime branching. Remove `IS_BUY` and hardcode the relevant swap path in each contract's `_buy` to make behavior explicit and minimize per-call gas checks. This simplifies initialization, reduces operator error risk, and improves auditability. Update deployment scripts/tests to instantiate the correct adapter type per strategy.

Note

The Diffuse Protocol team informed Hashlock that this issue will not be addressed in the current version. The protocol relies on view-only methods implemented in the `Viewer` contract to verify adapter chain integrity and ensure that tokens are ordered correctly.

Status

Acknowledged

[Q-02] Adapter#__Adapter_init - Missing Token Pair Distinctness Validation

Description

Adapter.__Adapter_init only checks that TOKEN_IN and TOKEN_OUT are non-zero before storing them.

It does not prevent initializing an adapter with the same token for both sides, which can happen due to configuration mistakes or reversed inputs.

When `TOKEN_IN == TOKEN_OUT`, swaps become no-ops and can lead to confusing behavior, invalid minAmountOut expectations, or wasted gas in production routes.

Recommendation

Add a guard in __Adapter_init to reject identical tokens and return a clear error.

Introduce a dedicated error (e.g., ErrorLib.IdenticalTokens) or reuse an existing InvalidToken error to keep revert reasons explicit.

Add a regression test that initializes an adapter with identical tokens and expects a revert.

Note

The Diffuse Protocol team informed Hashlock that simulation methods are implemented in the Viewer contract to manually detect potential confusion across adapter chains. Additionally, the team noted that supporting adapters with identical token pairs may be intentional in certain scenarios, such as claiming rewards within more complex integrations.

Status

Acknowledged

Centralisation

The Diffuse Protocol project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Diffuse Protocol project seems to have a sound and well-tested code base, now that our vulnerability findings have been acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.